Module 3 React Fundamentals

PROF. SUNITA D RATHOD

REACT

- React is a popular JavaScript library for building user interfaces, particularly for single-page applications.

- It allows developers to create reusable UI components and manage the state of their applications effectively.

INSTALLATION

Installing React involves setting up a development environment with the necessary tools to create and run React applications. Here are the steps to get started:

Prerequisites

Before installing React, you need to have Node.js and npm (Node Package Manager) installed on your system.

You can download and install Node.js from nodejs.org.

npm is included with Node.js.

You require one text editor , so for this you can use visual studio

Using Create React App

The easiest way to create a new React project is by using the Create React App CLI tool. It sets up a modern React project with sensible defaults and a good development experience.

1. **Install Create React App:**
   Open your terminal and run the following command to install Create React App globally:

npm install -g create-react-app

2. Create a New React Application:

Run the following command to create a new React project called `my-app` (you can replace `my-app` with your desired project name):

```
create-react-app my-app
```

This will create a new directory called **`my-app`** with all the necessary files and dependencies to start a React application.

3. Navigate to the Project Directory:

cd my-app

4. Start the Development Server:

Run the following command to start the development server:

npm start

Verify Installation

Open a command prompt (or PowerShell), and enter the following:

1. node –v : for displaying installed version of Node.js
2. npm –v : for displaying installed version of NPM .

FEATURES OF REACT

React is a popular JavaScript library for building user interfaces, particularly single-page applications where data can change over time without requiring a page reload.

Here are some of the key features that make React a powerful and widely used tool:

FEATURES OF REACT

1. JSX (JavaScript XML)

JSX is a syntax extension that allows you to write HTML-like code within JavaScript. This makes the code more readable and easier to write, as you can use familiar HTML syntax to create React components.

const element = <h1>Hello, world!</h1>;

FEATURES OF REACT

2. Components

Components are the building blocks of a React application. They encapsulate logic and UI into reusable pieces. Components can be either class-based or functional.

```
function Greeting(props) {

return <h1>Hello, {props.name}!</h1>;

 }
```

FEATURES OF REACT

3. Virtual DOM

React uses a virtual DOM to optimize updates. Instead of manipulating the real DOM directly, React creates a virtual representation of the DOM and updates it first. React then efficiently updates the real DOM to match the virtual DOM.

FEATURES OF REACT

4. One-Way Data Binding

Data in React flows in one direction, from parent to child components. This makes it easier to understand how data changes in the application and helps maintain a predictable data flow.

FEATURES OF REACT

5. Performance

react uses virtual DOM and updates only the modified parts. So , this makes the DOM to run faster. DOM executes in memory so we can create separate components which makes the DOM run faster.

FEATURES OF REACT

6. **Extension**

React has many extensions that we can use to create full-fledged UI applications. It supports mobile app development and provides server-side rendering. React is extended with Flux, Redux, React Native, etc. which helps us to create good-looking UI.

FEATURES OF REACT

7. **Simplicity:**

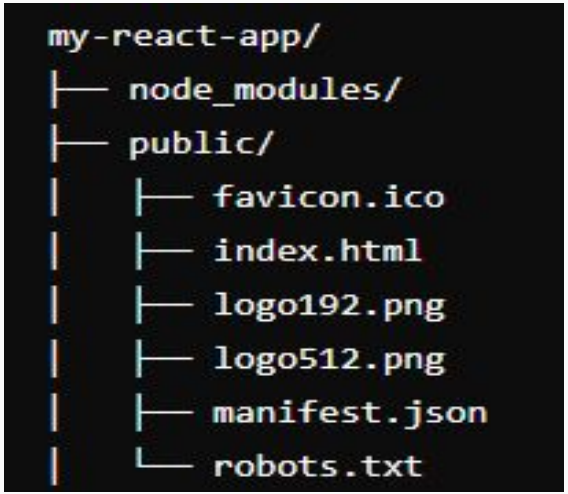
React.js is a component-based which makes the code reusable and React.js uses JSX which is a combination of HTML and JavaScript. This makes code easy to understand and easy to debug and has less code.

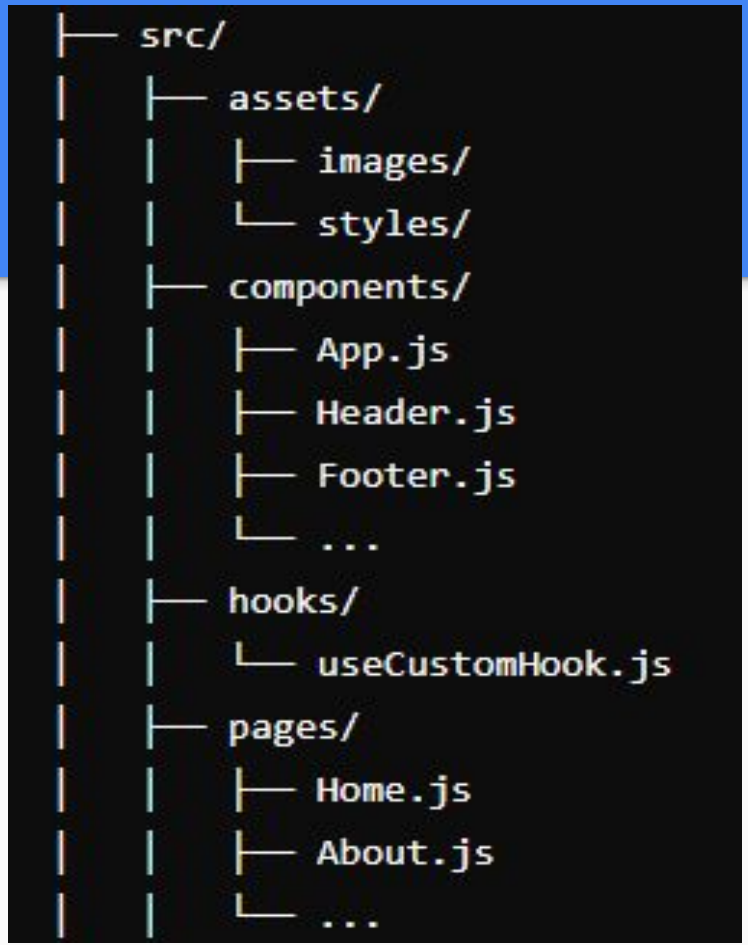
ADVANTAGE OF REACT

- It is composable.
- It is declarative.
- Write once, and learn anywhere.
- It is simple.
- SEO friendly.
- Fast, efficient, and easy to learn.
- It guarantees stable code.
- It is backed by a strong community.

React project File structure

A typical React project has a file structure that helps organize the codebase efficiently. Below is an example of a standard React project file structure:




src/
assets/
images/
styles/
components/
App.js
Header.js
Footer.js
...
hooks/
useCustomHook.js
pages/
Home.js
About.js
...


services/
api.js
utils/
helpers.js
App.js
App.test.js
index.js
index.css
reportWebVitals.js
setupTests.js
.gitignore
package.json
README.md
yarn.lock (or package-lock.json)

Explanation of Key Directories and Files:

- **node_modules/**: Contains all the project dependencies.
- **public/**: Contains static files that are served directly. The `index.html` file is the main HTML file, and other files like `favicon.ico` and images can also be placed here.
- **src/**: Contains the source code of the React application.
  - **assets/**: Contains static assets like images and CSS files.
    - **images/**: Directory for images.
    - **styles/**: Directory for CSS files or styled-components.

Explanation of Key Directories and Files:

- **components/**: Contains reusable components used across the application.
- **hooks/**: Custom React hooks.
- **pages/**: Components representing different pages of the application.
- **services/**: Contains services for API calls and other utilities.
- **utils/**: Contains utility functions.
- **App.js**: The root component of the application.
- **App.test.js**: Tests for the App component.
- **index.js**: The entry point of the application.
- **index.css**: Global CSS file.

Explanation of Key Directories and Files:

- **reportWebVitals.js**: For measuring performance.
- **setupTests.js**: Configuration for testing.

- **.gitignore**: Specifies which files and directories to ignore in version control.
- **package.json**: Lists the project dependencies and scripts.
- **README.md**: Contains information about the project.
- **yarn.lock** or **package-lock.json**: Lock file for dependencies.

Thank you